Пример 16. Стратегия (Strategy).

```
# include <iostream>
# include <memory>
# include <vector>

using namespace std;

class Strategy
{
public:
	virtual ~Strategy() = 0;

	virtual void algorithm() = 0;
};

Strategy::~Strategy() = default;

class ConStrategy1 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 1;" << endl; }
};

class ConStrategy2 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 2;" << endl; }
};

class Context
{
private:
	unique_ptr<Strategy> strategy;

public:
	explicit Context(Strategy* ptr) : strategy(ptr) {}

	void algorithmStrategy() { strategy->algorithm(); }
};

int main()
{
	Context obj(new ConStrategy1());

	obj.algorithmStrategy();
}
```

Пример 17. Стратегия (Strategy).

```
# include <iostream>
# include <memory>
# include <vector>

using namespace std;

class Strategy
{
public:
	virtual ~Strategy() = 0;

	virtual void algorithm() = 0;
};

Strategy::~Strategy() = default;

class ConStrategy1 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 1;" << endl; }
};
```

```
class ConStrategy2 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 2;" << endl; }
};

class Context
{
public:

	void algorithmStrategy(shared_ptr<Strategy> strategy) { strategy->algorithm(); }
};

int main()
{
	Context obj;

	obj.algorithmStrategy(shared_ptr<Strategy>(new ConStrategy1()));

	obj.algorithmStrategy(shared_ptr<Strategy>(new ConStrategy2()));
}
```

Пример 18. Стратегия (Strategy). “Статический полиморфизм”.

```
# include <iostream>
# include <memory>
# include <vector>

using namespace std;

class Strategy
{
public:
	virtual ~Strategy() = 0;

	virtual void algorithm() = 0;
};

Strategy::~Strategy() = default;

class ConStrategy1 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 1;" << endl; }
};

class ConStrategy2 : public Strategy
{
public:
	virtual void algorithm() override { cout << "Algorithm 2;" << endl; }
};

template <typename CStrategy>
class Context
{
private:
	unique_ptr<CStrategy> strategy;

public:
	Context() : strategy(new CStrategy()) {}

	void algorithmStrategy() { strategy->algorithm(); }
};

int main()
{
	Context<ConStrategy1> obj;

	obj.algorithmStrategy();
}
```

Пример 19. Команда (Command).

```
# include <iostream>
# include <memory>

using namespace std;

class Command
{
public:
	virtual ~Command() = default;
	virtual void execute() = 0;
};

template <typename Reseiver>
class SimpleCommand : public Command
{
private:
	typedef void(Reseiver::*Action)();

	shared_ptr<Reseiver> reseiver;
	Action action;

public:
	SimpleCommand(shared_ptr<Reseiver> r, Action a) : reseiver(r), action(a) {}

	virtual void execute() override { ((*reseiver).*action)(); }
};

class Object
{
public:
	void run() { cout << "Run method;" << endl; }
};

int main()
{
	shared_ptr<Object> obj(new Object());
	shared_ptr<Command> command(new SimpleCommand<Object>(obj, &Object::run));

	command->execute();
}
```

Пример 20. Цепочка обязанностей (Chain of Responsibility).

```
# include <iostream>
# include <memory>

using namespace std;

class AbstractHandler
{
protected:
	shared_ptr<AbstractHandler> next;

	virtual bool run() = 0;

public:
	using Default = shared_ptr<AbstractHandler>;

	virtual ~AbstractHandler() = default;

	virtual bool handle() = 0;

	void add(shared_ptr<AbstractHandler> node);
	void add(shared_ptr<AbstractHandler> node1, shared_ptr<AbstractHandler> node2, ...);
};

class ConHandler : public AbstractHandler
{
private:
	bool condition{ false };
```

```
protected:
	virtual bool run() override { cout << "Method run;" << endl; return true; }

public:
	ConHandler() = default;
	ConHandler(bool c) : condition(c) { cout << "Constructor;" << endl; }
	virtual ~ConHandler() override { cout << "Destructor;" << endl; }

	virtual bool handle() override
	{
		if (!condition) return next ? next->handle() : false;

		return run();
	}

};

#pragma region Methods
void AbstractHandler::add(shared_ptr<AbstractHandler> node)
{
	if (next)
		next->add(node);
	else
		next = node;
}

void AbstractHandler::add(shared_ptr<AbstractHandler> node1, shared_ptr<AbstractHandler> node2, ...)
{
	add(node1);
	for (Default* ptr = &node2; *ptr; ++ptr)
		add(*ptr);
}

#pragma endregion

int main()
{
	shared_ptr<AbstractHandler> chain(new ConHandler());

	chain->add(
		shared_ptr<AbstractHandler>(new ConHandler(false)),
		shared_ptr<AbstractHandler>(new ConHandler(true)),
		shared_ptr<AbstractHandler>(new ConHandler(true)),
		AbstractHandler::Default()
	);

	cout << "Result = " << chain->handle() << ";" << endl;;
}
```

Пример 21. Свойство (Property).

```
# include <iostream>
# include <memory>

using namespace std;

template <typename Owner, typename Type>
class Property
{
private:
	using Getter = Type (Owner::*)() const;
	using Setter = void (Owner::*)(const Type&);

	Owner* owner;
	Getter methodGet;
	Setter methodSet;

public:
	Property() = default;
	Property(Owner* owr, Getter getmethod, Setter setmethod) : owner(owr), methodGet(getmethod), methodSet(setmethod) {}
```

```
        void init(Owner* owr, Getter getmethod, Setter setmethod)
        {
                owner = owr;
                methodGet = getmethod;
                methodSet = setmethod;
        }

        operator Type() { return (owner->*methodGet)();         }                                       // Getter
        void operator=(const Type& data) {   (owner->*methodSet)(data); }          // Setter

//      Property(const Property&) = delete;
//      Property& operator=(const Property&) = delete;
};

class Object
{
private:
        double value;

public:
        Object(double v) : value(v) { Value.init(this, &Object::getValue, &Object::setValue); }

        double getValue() const { return value; }
        void setValue(const double& v) { value = v; }

        Property<Object, double> Value;
};

int main()
{
        Object obj(5.);

        cout << "value = " << obj.Value << endl;

        obj.Value = 10.;

        cout << "value = " << obj.Value << endl;

        unique_ptr<Object> ptr(new Object(15.));

        cout << "value =" << ptr->Value << endl;

        obj = *ptr;
        obj.Value = ptr->Value;
}
```

Пример 22. Посетитель (Visitor).

```
# include <iostream>
# include <memory>
# include <vector>

using namespace std;

class Circle;
class Rectangle;

class Visitor
{
public:
        virtual ~Visitor() = default;

        virtual void visit(Circle& ref) = 0;
        virtual void visit(Rectangle& ref) = 0;
};

class Shape
{
public:
        virtual ~Shape() = default;

        virtual void accept(shared_ptr<Visitor> visitor) = 0;
```

```
};

class Circle : public Shape
{
public:
	virtual void accept(shared_ptr<Visitor> visitor)  override { visitor->visit(*this);	}
};

class Rectangle : public Shape
{
public:
	virtual void accept(shared_ptr<Visitor> visitor)  override { visitor->visit(*this);	}
};

class ConVisitor : public Visitor
{
public:
	virtual void visit(Circle& ref) override { cout << "Circle;" << endl; }
	virtual void visit(Rectangle& ref) override { cout << "Rectangle;" << endl; }
};

class Formation
{
public:
	static vector<shared_ptr<Shape>> initialization(shared_ptr<Shape> elem, ...)
	{
		vector<shared_ptr<Shape>> vec;

		for (shared_ptr<Shape>* ptr = &elem; *ptr; ++ptr)
			vec.push_back(*ptr);

		return vec;
	}
};

int main()
{
	vector<shared_ptr<Shape>> figure = Formation::initialization(
		shared_ptr<Shape>(new Circle()),
		shared_ptr<Shape>(new Rectangle()),
		shared_ptr<Shape>(new Circle()),
		shared_ptr<Shape>()
	);
	shared_ptr<Visitor> visitor(new ConVisitor());

	for (auto& elem : figure)
		elem->accept(visitor);
}
```

Пример 23. Состояние (State).

```
# include <iostream>
# include <memory>
# include <string>

using namespace std;

class MusicPlayerState;

enum class State
{
	ST_STOPPED, ST_PLAYING, ST_PAUSED
};

class MusicPlayer
{
public:
	explicit MusicPlayer(MusicPlayerState* ptr) : pState(ptr) {}
	virtual ~MusicPlayer() = default;

	void Play();
	void Pause();
```

```
	void Stop();

	void SetState(State state);

private:
	unique_ptr<MusicPlayerState> pState;
};

class MusicPlayerState
{
public:
	MusicPlayerState(string nm) : name(nm) {}
	virtual ~MusicPlayerState() = 0;

	virtual void Play(MusicPlayer& player) {}
	virtual void Pause(MusicPlayer& player) {}
	virtual void Stop(MusicPlayer& player) {}

	string GetName() { return name; }

private:
	string   name;
};

MusicPlayerState::~MusicPlayerState() = default;

class PlayingState : public MusicPlayerState {
public:
	PlayingState() : MusicPlayerState(string("Playing")) {}
	virtual ~PlayingState() = default;

	virtual void Pause(MusicPlayer& player) override {	player.SetState(State::ST_PAUSED); }
	virtual void Stop(MusicPlayer& player) override { player.SetState(State::ST_STOPPED); }
};

class PausedState : public MusicPlayerState {
public:
	PausedState() : MusicPlayerState(string("Paused")) {}
	virtual ~PausedState() = default;

	virtual void Play(MusicPlayer& player) override { player.SetState(State::ST_PLAYING); }
	virtual void Stop(MusicPlayer& player) override { player.SetState(State::ST_STOPPED); }
};

class StoppedState : public MusicPlayerState {
public:
	StoppedState() : MusicPlayerState(string("Stopped")) {}
	virtual ~StoppedState() = default;

	virtual void Play(MusicPlayer& player) override { player.SetState(State::ST_PLAYING); }
};

#pragma region Methods MusicPlayer
void MusicPlayer::Play() { pState->Play(*this); }
void MusicPlayer::Pause() { pState->Pause(*this); }
void MusicPlayer::Stop() { pState->Stop(*this); }

void MusicPlayer::SetState(State state)
{
	cout << "changing from " << pState->GetName() << " to ";

	if (state == State::ST_STOPPED)
	{
		pState.reset(new StoppedState());
	}
	else if (state == State::ST_PLAYING)
	{
		pState.reset(new PlayingState());
	}
	else
	{
```

```
                pState.reset(new PausedState());
        }

        cout << pState->GetName() << " state" << endl;
}

#pragma endregion

int main()
{
        MusicPlayer player(new StoppedState());

        player.Play();
        player.Pause();
        player.Stop();
}
```

Пример 24. Посредник (Mediator).

```
# include <iostream>
# include <memory>
# include <list>
# include <vector>

using namespace std;

class Message {};       // Request

class Mediator;

class Colleague
{
private:
        weak_ptr<Mediator> mediator;

public:
        virtual ~Colleague() = default;

        void setMediator(shared_ptr<Mediator> mdr) { mediator = mdr; }

        virtual bool send(shared_ptr<Message> msg);
        virtual void receive(shared_ptr<Message> msg) = 0;
};

class ColleagueLeft : public Colleague
{
public:
        virtual void receive(shared_ptr<Message> msg) override { cout << "Right - > Left;" << endl; }
};

class ColleagueRight : public Colleague
{
public:
        virtual void receive(shared_ptr<Message> msg) override { cout << "Left - > Right;" << endl; }
};

class Mediator
{
protected:
        list<shared_ptr<Colleague>> colleagues;

public:
        virtual ~Mediator() = default;

        virtual bool send(const Colleague* coleague, shared_ptr<Message> msg) = 0;

        static bool add(shared_ptr<Mediator> mediator, shared_ptr<Colleague> colleague, ...);
};

class ConMediator : public Mediator
{
public:
```

```
	virtual bool send(const Colleague* coleague, shared_ptr<Message> msg) override;
};

#pragma region Methods Colleague
bool Colleague::send(shared_ptr<Message> msg)
{
	shared_ptr<Mediator> mdr = mediator.lock();

	return mdr ? mdr->send(this, msg) : false;
}

#pragma endregion

#pragma region Methods Mediator
bool Mediator::add(shared_ptr<Mediator> mediator, shared_ptr<Colleague> colleague, ...)
{
	if (!mediator || !colleague) return false;

	for (shared_ptr<Colleague>* ptr = &colleague; *ptr; ++ptr)
	{
		mediator->colleagues.push_back(*ptr);
		(*ptr)->setMediator(mediator);
	}

	return true;
}

bool ConMediator::send(const Colleague* colleague, shared_ptr<Message> msg)
{
	bool flag = false;
	for (auto& elem : colleagues)
	{
		if (dynamic_cast<const ColleagueLeft*>(colleague) && dynamic_cast<ColleagueRight*>(elem.get()))
		{
			elem->receive(msg);
			flag = true;
		}
		else if (dynamic_cast<const ColleagueRight*>(colleague) && dynamic_cast<ColleagueLeft*>(elem.get()))
		{
			elem->receive(msg);
			flag = true;
		}
	}

	return flag;
}

#pragma endregion

int main()
{
	shared_ptr<Mediator> mediator(new ConMediator());

	shared_ptr<Colleague> col1(new ColleagueLeft());
	shared_ptr<Colleague> col2(new ColleagueRight());
	shared_ptr<Colleague> col3(new ColleagueLeft());
	shared_ptr<Colleague> col4(new ColleagueLeft());

	Mediator::add(mediator, col1, col2, col3, col4, shared_ptr<Colleague>());

	shared_ptr<Message> msg(new Message());

	col1->send(msg);
	col2->send(msg);
}
```

Пример 25. Подписчик-издатель (Publish-Subscribe).

```
# include <iostream>
# include <memory>
# include <vector>
```

```
using namespace std;

class Subscriber;

using Reseiver = Subscriber;

class Publisher
{
private:
	using Action = void(Reseiver::*)();
	using Pair = pair<shared_ptr<Reseiver>, Action>;

	vector<Pair> callback;

public:
	void subscribe(shared_ptr<Reseiver> r, Action a);

	void run();
};

class Subscriber
{
public:
	virtual ~Subscriber() = default;

	virtual void method() = 0;
};

class ConSubscriber : public Subscriber
{
public:
	virtual void method() override { cout << "method;" << endl; }
};

#pragma region Methods Publisher
void Publisher::subscribe(shared_ptr<Reseiver> r, Action a)
{
	Pair pr(r, a);

	callback.push_back(pr);
}

void Publisher::run()
{
	cout << "Run:" << endl;
	for (auto elem : callback)
		((*elem.first).*(elem.second))();
}

#pragma endregion

int main()
{
	shared_ptr<Subscriber> subscriber(new ConSubscriber());
	shared_ptr<Publisher> publisher(new Publisher());

	publisher->subscribe(subscriber, &Subscriber::method);

	publisher->run();
}
```

Пример 26. Опекун (Memento).

```
# include <iostream>
# include <memory>
# include <list>

using namespace std;

class Memento;

class Caretaker
```

```
{
public:
	unique_ptr<Memento> getMemento();
	void setMemento(unique_ptr<Memento> memento);

private:
	list<unique_ptr<Memento>> mementos;
};

class Originator
{
public:
	Originator(int s) : state(s) {}

	const int getState() const { return state; }
	void setState(int s) { state = s; }

	std::unique_ptr<Memento> createMemento() { return make_unique<Memento>(*this); }
	void restoreMemento(std::unique_ptr<Memento> memento);

private:
	int state;
};

class Memento
{
	friend class Originator;

public:
	Memento(Originator o) : originator(o) {}

private:
	void setOriginator(Originator o) { originator = o; }
	Originator getOriginator() { return originator; }

private:
	Originator originator;
};

#pragma region Methods Caretaker
void Caretaker::setMemento(unique_ptr<Memento> memento)
{
	mementos.push_back(move(memento));
}

unique_ptr<Memento> Caretaker::getMemento() {

	unique_ptr<Memento> last = move(mementos.back());

	mementos.pop_back();

	return last;
}

#pragma endregion

#pragma region Method Originator
void Originator::restoreMemento(std::unique_ptr<Memento> memento)
{
	*this = memento->getOriginator();
}

#pragma endregion

int main()
{
	auto originator = make_unique<Originator>(1);
	auto caretaker = make_unique<Caretaker>();

	cout << "State = " << originator->getState() << endl;
	caretaker->setMemento(originator->createMemento());
```

```
	originator->setState(2);
	cout << "State = " << originator->getState() << endl;
	caretaker->setMemento(originator->createMemento());
	originator->setState(3);
	cout << "State = " << originator->getState() << endl;
	caretaker->setMemento(originator->createMemento());

	originator->restoreMemento(caretaker->getMemento());
	cout << "State = " << originator->getState() << endl;
	originator->restoreMemento(caretaker->getMemento());
	cout << "State = " << originator->getState() << std::endl;
	originator->restoreMemento(caretaker->getMemento());
	cout << "State = " << originator->getState() << std::endl;
}
```

Пример 27. Шаблонный метод (Template Method).

```
# include <iostream>

using namespace std;

class AbstractClass
{
public:
	void templateMethod()
	{
		primitiveOperation();
		concreteOperation();
		hook();
	}

protected:
	virtual void primitiveOperation() = 0;
	void concreteOperation() { cout << "concreteOperation;" << endl; }
	virtual void hook() { cout << "hook Base;" << endl; }
};

class ConClassA : public AbstractClass
{
protected:
	virtual void primitiveOperation() override { cout << "primitiveOperation A;" << endl; }
};

class ConClassB : public AbstractClass
{
protected:
	virtual void primitiveOperation() override {	cout << "primitiveOperation B;" << endl; }
	void hook() { cout << "hook B;" << endl; }
};

int main()
{
	ConClassA ca;
	ConClassB cb;
	ca.templateMethod();
	cb.templateMethod();
}
```